Встроенный искусственный интеллект с использованием MATLAB и Simulink

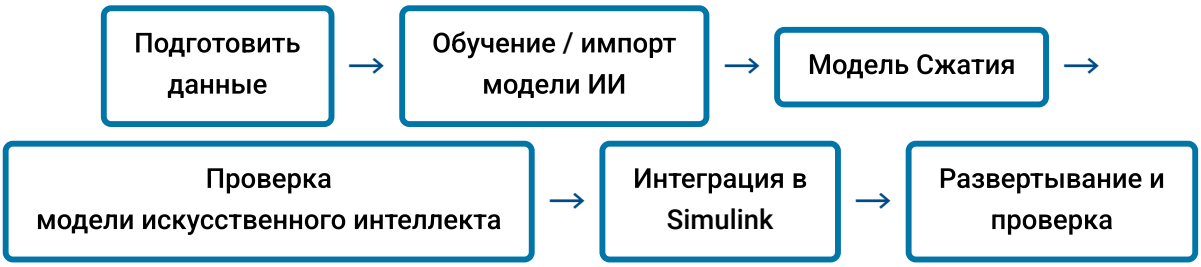
От концепции до производства — развертывайте ИИ на любом встроенном оборудовании.

Зачем нужен встроенный ИИ с MATLAB и Simulink?

Развертывайте обученные модели искусственного интеллекта на аппаратных средствах с ограниченными ресурсами — микроконтроллерах, графических процессорах, ПЛИС и нейронных процессорах — с помощью моделирования на системном уровне и автоматической генерации кода.

- **Моделирование на системном уровне:** тестируйте поведение ИИ вместе с контроллерами, датчиками и моделями оборудования, прежде чем приступать к работе с аппаратным обеспечением.
- **Генерация кода:** генерируйте оптимизированный код на C/C++, CUDA или HDL непосредственно из вашей модели Simulink, включая компонент ИИ, без ручного переноса.
- **Гибкость импорта:** используйте модели PyTorch, ONNX или TensorFlow и развертывайте их с помощью одного и того же конвейера.
- **Проверка на всех этапах.** Проверяйте свой ИИ-компонент на каждом этапе с помощью формальных методов, тестирования на устойчивость к состязательным атакам, а также тестирования «программное обеспечение в контуре» (software-in-the-loop, SIL), «процессор в контуре» (processor-in-the-loop, PIL) и «аппаратное обеспечение в контуре» (hardware-in-the-loop, HIL).
- **Соответствие стандартам:** Создавайте код, соответствующий стандарту MISRA C, с возможностью отслеживания для поддержки сертификации по стандартам DO-178C, ISO 26262 и IEC 61508.

Комплексный встроенный рабочий процесс с использованием искусственного интеллекта



Итеративный по своей природе:

Этот рабочий процесс не является строго линейным. Шаги могут повторяться, меняться местами или полностью пропускаться в зависимости от ограничений по памяти, требований к задержк целевого оборудования и соответствия стандартам.

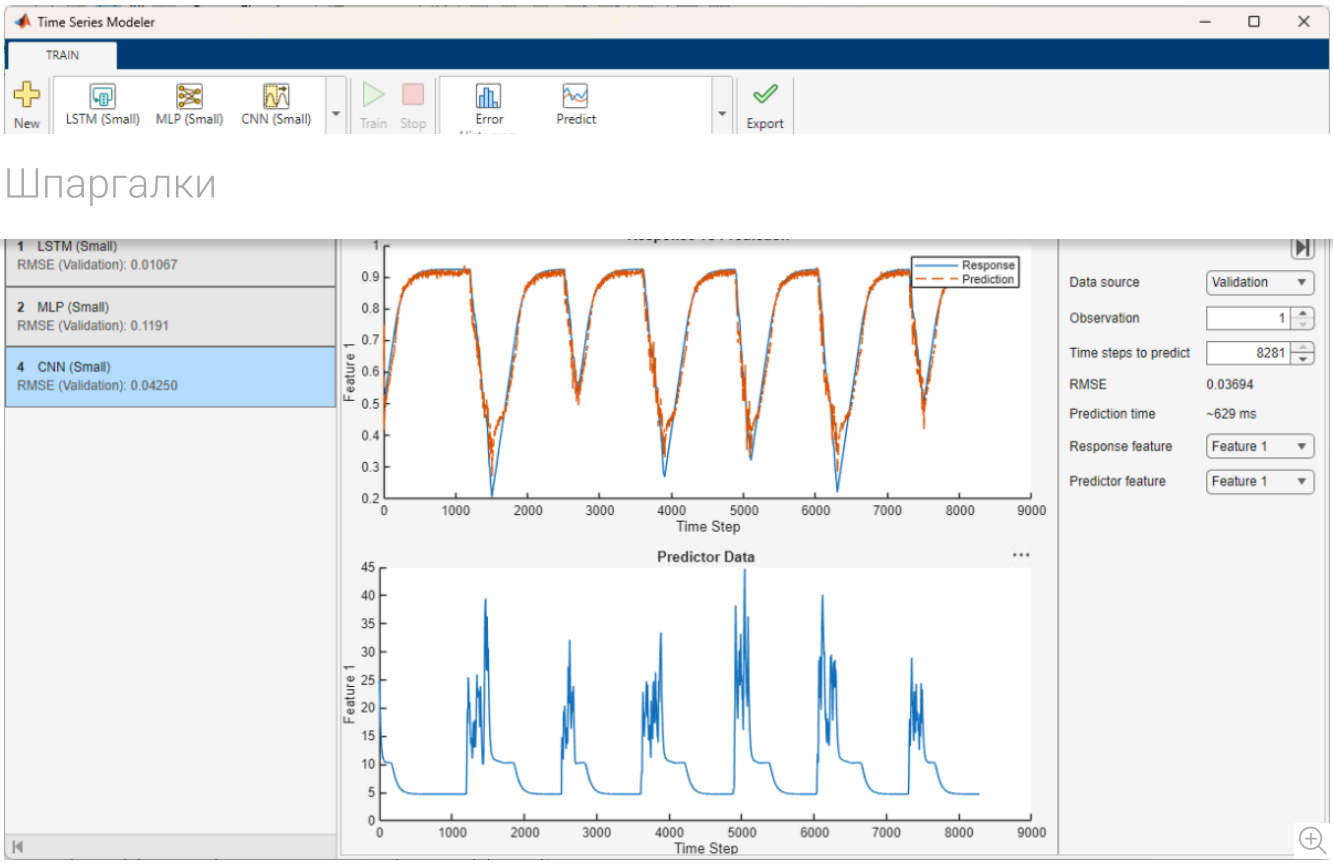
Обучать / Импортировать Модель искусственного Интеллекта

Программно обучайтесь в MATLAB

```
% Обучить сеть глубокого обучения
net = trainnet(data, layers, "crossentropy", options);

% Обучить модель машинного обучения
mdl = fitcsvm(features, labels);
```

Интерактивное обучение в MATLAB



Шпаргалки

- Конструктор глубоких нейронных сетей
- Модель временных рядов
- Обучение классификации
- Обучение регрессии

Импорт из внешних фреймворков

```
% Импортировать из PyTorch (экспортированный формат программы)
net = importNetworkFromPyTorch("exported_pytorch_model.pt2")

net =
 dlnetwork со свойствами:

 Слои: [9×1 nnet.cnn.layer.Layer]
 Соединения: [таблица 11×2]
 Обучаемые параметры: [таблица 86×3]
```

```
Состояние: [таблица 42×3]
InputNames: {'InputLayer1'}
OutputNames: {'ResidualNetSmall:fc'}
Инициализировано: 1
```

Источник	Функция
PyTorch (.pt2/.pt)	`importNetworkFromPyTorch`
ONNX	`importNetworkFromONNX`
Тензорный поток 2	`importNetworkFromTensorFlow`
Keras 3	`importNetworkFromKeras`
XGBoost (.json)	`importModelFromXGBoost`

Модель сжатия

```
%% Шаг 1: обрезка (например, удаление 60 % обучаемых элементов)
netPruned = compressNetworkUsingTaylorPruning(net, dsTrain, "crossentropy", ...
 options, LearnablesReductionGoal=0.6);

%% Шаг 2: проецирование (например, сохранение 80 % дисперсии)
npca = neuronPCA(netPruned, dsTrain);
netProjected = compressNetworkUsingProjection(netPruned, npca, ...
 ExplainedVarianceGoal=0.8);
netProjected = trainnet(data, netProjected, "crossentropy", optionsFT);

%% Шаг 3: квантизация (INT8)
quantObj = dlquantizer(netProjected, ExecutionEnvironment="CPU");
calibrate(quantObj, dsCal);
netQuantized = квантование(quantObj);
```

Техника	Потенциальное уменьшение размера модели	Когда использовать
Обрезка	50–70%	Сверточные нейронные сети с избыточными фильтрами
Проекция	20–85%	Сети с большим количеством полносвязных или рекуррентных связей и коррелированными активациями
Квантование	75% (4×)	Последний шаг для процессоров с фиксированной точкой

Подсказка:

Рекомендуемый порядок: обрезка → проектирование → квантование (тонкая настройка после каждого этапа). Используйте `estimateNetworkMetrics(net)` для измерения обучаемости, памя

активации и MAC-адресов до и после каждого этапа. См. также библиотеку сжатия моделей Deep Learning Toolbox.

Проверка модели искусственного Интеллекта

Докажите безопасность или оцените надежность *до* развертывания с помощью библиотеки для верификации ИИ и интерфейса Deep Learning Toolbox для альфа-бета-верификатора CROWN. В отличие от тестирования на выборочных входных данных, формальная верификация дает математические гарантии для непрерывных областей входных данных.

Техника	Что он Делает	Ключевая функция
Проверка надежности	Доказывает, что классификация сети инвариантна в пределах ограниченной области входных данных	`verifyNetworkRobustness`
Формальные границы вывода	Вычисляет гарантированные верхние и нижние границы выходных данных сети для ограниченной области входных данных	`estimateNetworkOutputBounds`
Состязательная надежность	Находит состязательные примеры, которые приводят к ошибочной классификации в пределах ограниченной области входных данных.	`findAdversarialExamples`
Обнаружение вненормальных значений	Флаги для входных данных, в отличие от обучающих данных, предотвращают скрытые сбои во время выполнения	`networkDistributionDiscriminator`

Первый аргумент — это либо объект `dlnetwork` (обученный в MATLAB или импортированный), либо путь к файлу модели — файлу ONNX (`.onnx`) или полной модели PyTorch (сохраненной с помощью `torch.save()`). Те же функции, тот же синтаксис.

```
% Доказать, что классификация устойчива к помехам датчиков в области входных данных X0
XLower = X0 - epsilon;
XUpper = X0 + epsilon;
[result, cex] = verifyNetworkRobustness(net, XLower, XUpper, trueLabel);

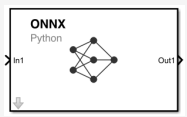
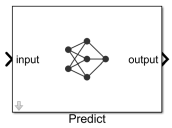
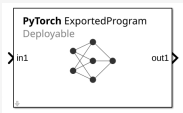
% Вычислить гарантированные границы выходных данных в области входных данных
```

```
[YLower, YUpper] = estimateNetworkOutputBounds(net, XLower, XUpper);

% Найти состязательные примеры в пределах ограниченной области
[adversarials, success] = findAdversarialExamples(net, XLower, XUpper, trueLabel);
```

Интеграция в Simulink

Встраивайте модели искусственного интеллекта в системное моделирование, чтобы проверять их поведение вместе с контроллерами, датчиками и моделями оборудования, прежде чем генерировать код.

Блок	Код	Пример использования
Совместное исполнение		Моделируйте PyTorch, TensorFlow, ONNX или пользовательские модели Python непосредственно в Simulink без преобразования; оценивайте работу сторонних ИИ-систем в рамках более крупных систем перед их полной интеграцией.
Предсказать		Запустите `dlnetwork` как единый блок логического вывода (классификация или регрессия)
Экспортированная программа PyTorch		Запустите модель PyTorch `.pt2` непосредственно в Simulink с помощью генерации кода на C/C++ и CUDA
Блоки слоев	`exportNetworkToSimulink`	Экспорт сетей в виде отдельных блоков Simulink для послойного управления и проверки

Развертывание и проверка

Сгенерируйте автономный исходный код, который будет работать без MATLAB, а затем постепенно тестируйте его на целевом оборудовании.

Генерация кода

Продукт	Выходной сигнал	Основные цели	Целевая Библиотека
Программист MATLAB	C/C++	ARM Cortex-A, x86, любая POSIX/RTOS	Автономный*, Intel oneDNN

Продукт	Выходной сигнал	Основные цели	Целевая Библиотека
Встроенный Кодер	Производственный C / C ++	NXP, Infineon, STMicro, микроконтроллеры Renesas и другие	Standalone*, CMSIS, CMSIS-NN
Графический Кодер	CUDA C++	NVIDIA Jetson Thor, Orin, Xavier, TX2	Standalone*, TensorRT
Встроенный кодер + HSP	Оптимизированный C/C++ для NPU	Qualcomm Hexagon, PPU Infineon (AURIX TC4x)	Vendor NPU runtime
HDL - Кодер	VHDL/Verilog	ПЛИС AMD (Xilinx), ПЛИС Intel	Deep Learning HDL Toolbox IP

*Установите целевую библиотеку глубокого обучения на `'none'` для создания автономного кода ANSI/ISO C/C++ для любого процессора без зависимости от сторонних библиотек.

Шаблон функции точки входа

```
functiondlnetwork
% Используйте MATLAB  out = myPredict(in) %#codegen
    persistent net
 if isempty(net)
 net = coder.loadDeepLearningNetwork('myNet.mat');
 end
 out = predict(net, in);
end
```

```
function
% Использовать модель PyTorch out = myPredict(in) %#codegen
    persistent pytorchNet
 if isempty(pytorchNet)
 pytorchNet = loadPyTorchExportedProgram('myPyTorchNet.pt2');
 end
 out = invoke(pytorchNet, in);
end
```

Настройка и генерация

```
% Сгенерировать C++ для любого процессора
cfg = coder.config('lib');
cfg.TargetLang = 'C++';
cfg.DeepLearningConfig = coder.DeepLearningConfig('none');
codegen -config cfg myPredict -args {ones(224,224,3,'single')}

% Сгенерировать CUDA для NVIDIA Jetson
gpuCfg = coder.gpuConfig('lib');
gpuCfg.DeepLearningConfig = coder.DeepLearningConfig('tensorrt');
codegen -config gpuCfg myPredict -args {ones(224,224,3,'single')}
```

Целевая Библиотека	Аппаратное обеспечение
`'none'`	Любой (без использования библиотеки)
`'mkldnn'`	x86-64 (Intel oneDNN)
`'cudnn'`	Графические процессоры NVIDIA
`'tensorrt'`	Графические процессоры NVIDIA/Jetson

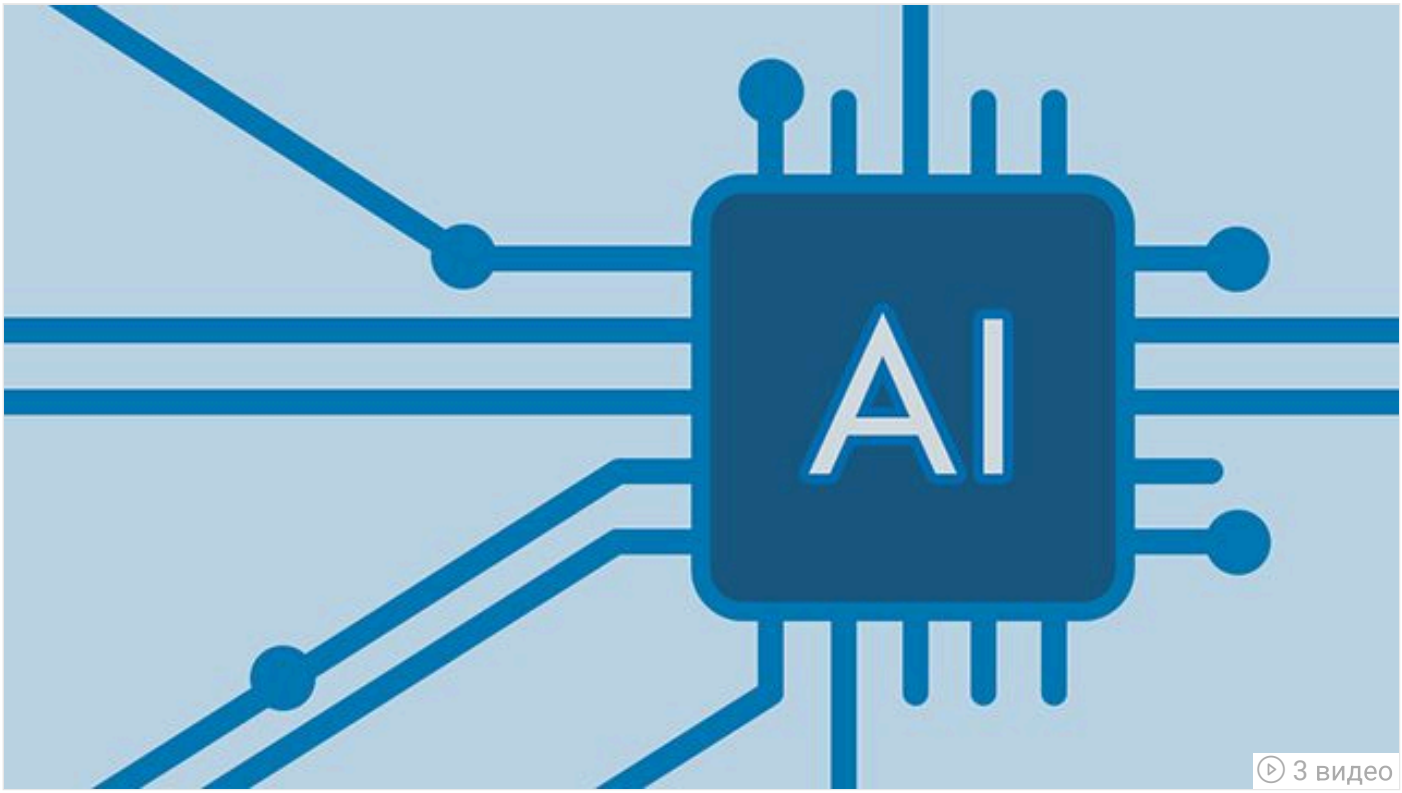
Проверка на системном уровне (MIL/SIL/PIL/HIL)

Поэтапная проверка: модель (**MIL**) → сгенерированный код на хосте (**SIL**) → целевой процессор (**PIL**) → вся система с реальным вводом-выводом (**HIL**).

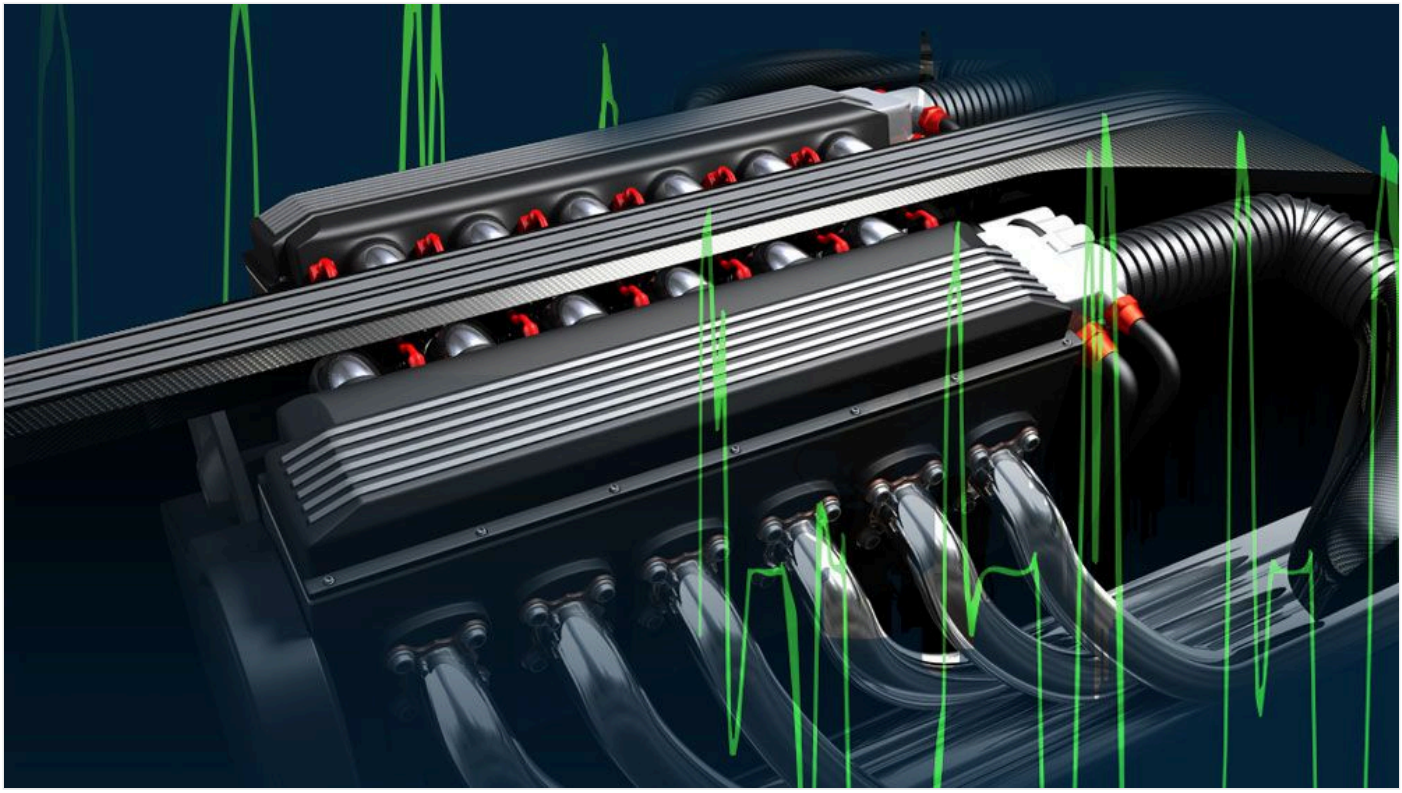
Этап	Что Бежит	Где	Что Это Подтверждает
MIL (интеллектуальная система управления)	Модель Simulink (интерпретируемая)	Хост-ПК	Корректность алгоритма: создание эталонного образца
SIL (программное управление)	Сгенерированный код на C/C++/CUDA	Главный компьютер (скомпилированный)	Поведенческая корректность: численная эквивалентность сгенерированного кода, выполняемого на главном процессоре
PIL (процессор в контуре управления)	Сгенерированный код на C/C++/CUDA	Целевое оборудование	Эффекты, зависящие от целевого устройства: компилятор, FPU, численная эквивалентность сгенерированного кода, работающего на целевом процессоре
HIL (аппаратно-программный комплекс)	Полная система с реальными операциями ввода-вывода	Цель в реальном времени	Real-time effects: integration, timing, and I/O behavior

```
% Проверка Processor-in-the-Loop
set_param("myModel/AI_Subsystem", "SimulationMode", "Processor-in-the-loop");
out = sim("myModel");
% Сравните выходные данные PIL с базовыми данными MIL, чтобы выявить численный дрейф
```

Продолжайте Исследовать



Edge и встроенный искусственный интеллект (3 видео)



ИИ с проектированием на основе моделей



Интеграция ИИ в проектирование на системном уровне: использование ИИ для встроенных алгоритмов